LAB 2:

```
import csv
with open("training_lab2.csv") as f:
    data = list(csv.DictReader(f))
A = ["Sky", "AirTemp", "Humidity", "Wind", "Water", "Forecast"]
S = ["0"] * 6
G = [["?"] * 6]
def covers(h, x):
    return all(h[i] == "?" or h[i] == x[A[i]] for i in range(6))

for n, x in enumerate(data, 1):

    if x["EnjoySport"] == "Yes":
        for i in range(6):
            if S[i] == "0":
                S[i] = x[A[i]]
            elif S[i] != x[A[i]]:
                S[i] = "?"
        G = [h for h in G if covers(h, x)]
    else:
        new_G = []
        for h in G:
            if covers(h, x):
                for i in range(6):
                    if h[i] == "?":
                        new_h = h.copy()
                        new_h[i] = S[i]
                        if new_h[i] != "Ø":
                            new_G.append(new_h)
            else:
                new_G.append(h)
        G = new_G

    print(f"\nExample {n}: {x['EnjoySport']}")
    print("S =", S)
    print("G =", G)

print("\n========== FINAL RESULT ==========")
print("Specific Boundary (S):", S)
print("General Boundary (G):")
for h in G:
    print(h)
```

LAB 3:

```
import csv, math
from collections import Counter

with open("cricket.csv") as f:
    data = list(csv.DictReader(f))

T = "Play"
A = ["Weather", "Wind"]

def entropy(rows):
    c = Counter(r[T] for r in rows)
    n = len(rows)
```

```
    return -sum(v/n * math.log2(v/n) for v in c.values())

def id3(rows, attrs):
    y = [r[T] for r in rows]

    if len(set(y)) == 1:
        return y[0]
    if not attrs:
        return Counter(y).most_common(1)[0][0]

    best = max(attrs, key=lambda a: entropy(rows) -
        sum(len(s)/len(rows)*entropy(s)
        for v in set(r[a] for r in rows)
        for s in [[r for r in rows if r[a] == v]]))

    return {best: {v: id3(
        [r for r in rows if r[best] == v],
        [a for a in attrs if a != best])
        for v in set(r[best] for r in rows)}}

print("Decision Tree:")
print(id3(data, A))
```

LAB 4:

```
import csv
from collections import Counter
with open("naive_bayes_train.csv") as f:
    train = list(csv.DictReader(f))
with open("naive_bayes_test.csv") as f:
    test = list(csv.DictReader(f))
T = "PlayTennis"
F = ["Outlook", "Temperature", "Humidity", "Wind"]
C = Counter(r[T] for r in train)
N = len(train)
def predict(x):
    scores = {}
    for c in C:
        p = C[c] / N
        for f in F:
            n = sum(r[T] == c and r[f] == x[f] for r in train)
            k = len(set(r[f] for r in train))
            p *= (n + 1) / (C[c] + k)
        scores[c] = p
    return max(scores, key=scores.get)
correct = 0
for x in test:
    p = predict(x)
    print("Actual:", x[T], "Predicted:", p)
    correct += (x[T] == p)
print("Correct:", correct)
print("Total:", len(test))
print("Accuracy:", round(correct / len(test) * 100, 2), "%")
```

LAB 5:

```
from collections import Counter, defaultdict
from math import prod
```

```
data = [("cricket won", "Sports"),
          ("government bill", "Politics")]
test = ["cricket won", "government bill"]
actual = ["Sports", "Politics"]
C, W = Counter(), defaultdict(Counter)
for x, y in data:
     C[y] += 1
     for w in x.split():
          W[y][w] += 1
V = set(w for x, y in data for w in x.split())
def predict(x):
     return max(C, key=lambda c: C[c] * prod(
     (W[c][w]+1)/(sum(W[c].values())+len(V))
     for w in x.split()))
pred = [predict(x) for x in test]
for x, p in zip(test, pred):
     print(x, "=>", p)
TP = sum(a == p == "Sports" for a, p in zip(actual, pred))
TN = sum(a == p == "Politics" for a, p in zip(actual, pred))
FP = sum(a == "Politics" and p == "Sports" for a, p in zip(actual, pred))
FN = sum(a == "Sports" and p == "Politics" for a, p in zip(actual, pred))
print("TP =", TP, "TN =", TN, "FP =", FP, "FN =", FN)
print("Accuracy =", (TP+TN)/len(test))
print("Precision =", TP/(TP+FP) if TP+FP else 0)
print("Recall =", TP/(TP+FN) if TP+FN else 0)
```

LAB 6:

```
import csv

data = []

with open("heart.csv", "r") as file:
    reader = csv.DictReader(file)
    for row in reader:
        data.append(row)
print("Number of records:", len(data))

yes = 0
no = 0
for row in data:
    if row["HeartDisease"] == "Yes":
        yes += 1
    else:
        no += 1
p_yes = yes / len(data)
p_no = no / len(data)

patient = {"Age": "Old","Sex": "Male",
    "ChestPain": "Typical","BP": "High",
    "Cholesterol": "High","HeartRate": "Normal"}

def probability(attribute, value, disease):
    count = 0
    disease_count = 0

    for row in data:
        if row["HeartDisease"] == disease:
            disease_count += 1
            if row[attribute] == value:
                count += 1
```

```
    return count / disease_count

prob_yes = p_yes
for attribute in patient:
    prob_yes = prob_yes * probability(attribute,patient[attribute],"Yes")
prob_no = p_no
for attribute in patient:
    prob_no = prob_no * probability(attribute,patient[attribute],"No")

print("\nProbability of Heart Disease:", prob_yes)
print("Probability of No Heart Disease:", prob_no)

if prob_yes > prob_no:
    print("\nDiagnosis: Heart Disease")
else:
    print("\nDiagnosis: No Heart Disease”)

LAB 7:
import csv
import math
data = []
with open("students_Lab7.csv") as f:
    for row in csv.DictReader(f):
        data.append(float(row["Marks"]))
m1, m2 = 50, 90
for _ in range(10):
    r1, r2 = [], []
   for x in data:
        p1 = math.exp(-(x-m1)**2 / 200)
        p2 = math.exp(-(x-m2)**2 / 200)
        r1.append(p1/(p1+p2))
        r2.append(p2/(p1+p2))
   m1 = sum(r1[i]*data[i] for i in range(len(data))) / sum(r1)
    m2 = sum(r2[i]*data[i] for i in range(len(data))) / sum(r2)
for i, x in enumerate(data):
    cluster = 1 if r1[i] > r2[i] else 2
    print(x, "-> Cluster", cluster)
print("Means:", round(m1, 2), round(m2, 2))

LAB 8:
import csv
import math
data = []

with open("students_kmeans.csv") as file:
    reader = csv.reader(file)
    next(reader)

    for row in reader:
        data.append([float(row[1]), float(row[2])])
centroids = [[2, 45], [6, 65], [10, 90]]
for _ in range(5):

    clusters = [[], [], []]
   for point in data:
        distances = []

        for center in centroids:
            d = math.sqrt((point[0]-center[0])**2 +
                          (point[1]-center[1])**2)
```

```
            distances.append(d)

        cluster = distances.index(min(distances))
        clusters[cluster].append(point)

    for i in range(3):
        if clusters[i]:
            x = sum(p[0] for p in clusters[i]) / len(clusters[i])
            y = sum(p[1] for p in clusters[i]) / len(clusters[i])
            centroids[i] = [x, y]
for i in range(3):
    print("Cluster", i + 1)
    print(clusters[i])
    print("Centroid:", centroids[i])
```

LAB 9 :

```
import csv
import math
data = []

with open("iris.csv") as f:
    reader = csv.reader(f)
    next(reader)
    for row in reader:
        x = [float(v) for v in row[:4]]
        data.append([x, row[4]])
n = len(data)
train = data[:int(n * 0.75)]
test = data[int(n * 0.75):]
print("Total records:", len(data))
print("Training records:", len(train))
print("Testing records:", len(test))
K = 5
def predict(x):
    distances = []
    for row in train:
        dist = math.sqrt(sum((x[I] - row[0][i]) ** 2 for i in range(4)))
        distances.append((dist, row[1]))
   distances.sort()
   votes = {}
    for _, label in distances[:K]:
        votes[label] = votes.get(label, 0) + 1
   return max(votes, key=votes.get)
correct = 0
wrong = 0
print("\nPREDICTIONS")
print("----------------------------")
for row in test:
    actual = row[1]
    predicted = predict(row[0])
    if actual == predicted:
        correct += 1
        print("Correct :", actual, "->", predicted)
    else:
        wrong += 1
        print("Wrong   :", actual, "->", predicted)
print("\nRESULT")
```

```
print("-----------------------------")
print("Correct Predictions:", correct)
print("Wrong Predictions:", wrong)

if len(test) > 0:
    accuracy = correct / len(test) * 100
    print("Accuracy:", accuracy, "%")
else:
    print("No test data available.”)
```

LAB 10:

```
import csv
import math
import matplotlib.pyplot as plt

x = []
y = []

with open("study_hours_lab10.csv") as file:
    reader = csv.reader(file)
    next(reader)

    for row in reader:
        x.append(float(row[0]))
        y.append(float(row[1]))

def predict(x0, tau=1.0):
    weights = []
    for i in range(len(x)):
        w = math.exp(-((x[i] - x0) ** 2) / (2 * tau ** 2))
        weights.append(w)
   numerator = sum(weights[i] * y[i] for i in range(len(x)))
    denominator = sum(weights)
    return numerator / denominator

x_pred = []
y_pred = []

for value in x:
    x_pred.append(value)
    y_pred.append(predict(value))

plt.scatter(x, y, label="Actual Data")
plt.plot(x_pred, y_pred, label="LWR Curve")

plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Locally Weighted Regression")
plt.legend()
plt.show()
```